Machine Learning Operations (AIMLCZG523)

Assignment 01

Heart Disease Risk Prediction

End-to-end ML model development, CI/CD and production deployment

Shankaraguru Ramdurg

BITS ID 2025AE05523

M.Tech AI & ML (WILP), BITS Pilani

Code repository: https://github.com/<your-github-username>/heart-disease-mlops

October 2026

1. Summary

This project builds a classifier that predicts the presence of heart disease from 13 routine clinical measurements, and runs it as a monitored, containerised REST API on Kubernetes. Every stage is automated and reproducible: data acquisition with checksums, cleaning, EDA, tuned model training with full MLflow tracking and a model registry, 50 automated tests, a five-stage GitHub Actions pipeline, a hardened Docker image, Kubernetes manifests with autoscaling and zero-downtime rollouts, and Prometheus/Grafana monitoring. The same code also runs end to end inside BITS Prayogshala (Kubeflow) from a single notebook or as a Kubeflow Pipeline.

Item	Result
Data	All four UCI hospitals combined: 920 rows, 918 after de-duplication, 55.3% positive
Champion model	L1 logistic regression (C = 0.3) in a single sklearn Pipeline
5-fold CV ROC-AUC	0.905 ± 0.014
Hold-out test (184 patients)	ROC-AUC 0.899, recall 0.912, specificity 0.732, accuracy 0.832 at threshold 0.39
Experiment tracking	78 MLflow runs (pipeline, 3 model families, 72 grid candidates, ablation, champion), registered model heart-disease-classifier@champion
Quality	50 pytest tests, 97% line coverage, flake8 clean; clean-environment install verified
Deployment	Docker image (non-root, health check) on Kubernetes: Deployment (2-6 pods, HPA), LoadBalancer Service, Ingress, PDB
Monitoring	Prometheus metrics with 5 alert rules, Grafana dashboard (12 panels), JSON request logs, /monitoring page

2. Architecture

The system has three layers. The experimentation layer turns raw UCI files into a versioned, packaged model. The serving layer runs that model behind a validated API on Kubernetes, where it is scraped by Prometheus. The automation layer (GitHub Actions, plus Prayogshala/Kubeflow) re-runs everything on each change and blocks releases that fail a gate.

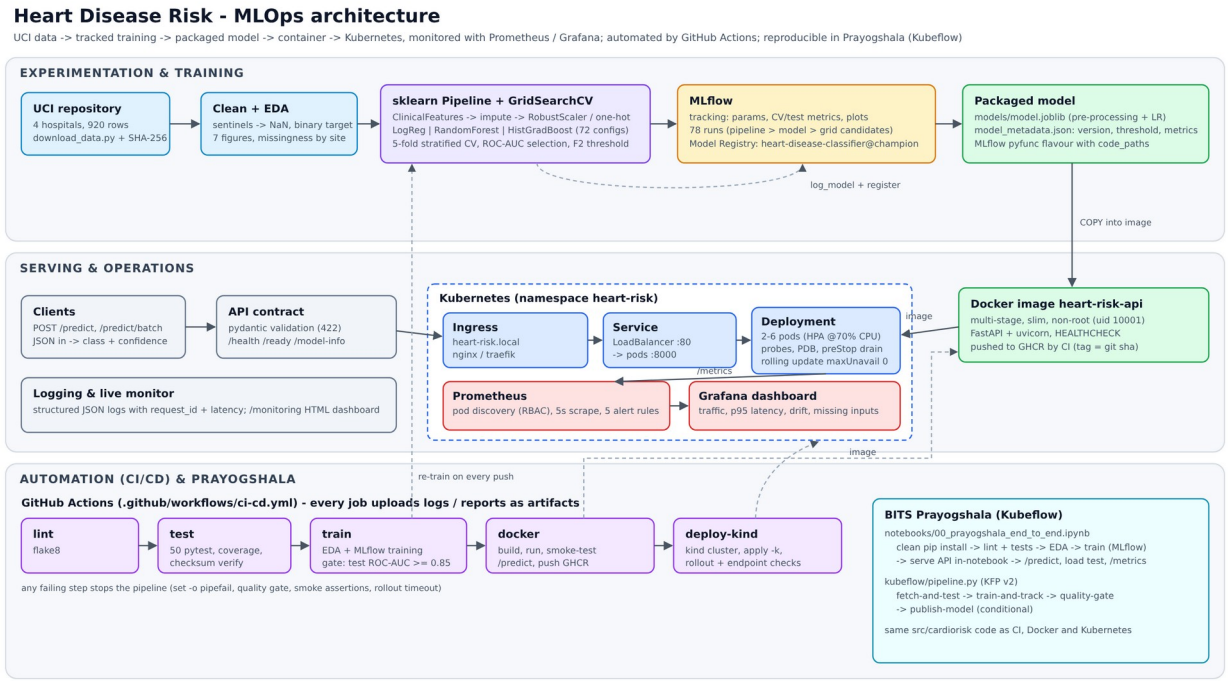


Figure 1 - System architecture

Key design principle: one artefact, one code path. Feature engineering, imputation, scaling and encoding live inside the saved sklearn Pipeline (models/model.joblib). The API, the MLflow pyfunc model, the notebooks and the Kubeflow pipeline all import the same src/cardiorisk package, so there is no training/serving skew.

3. Data acquisition and EDA

3.1 Acquisition

scripts/download_data.py downloads the UCI archive (dataset 45), falling back to the legacy per-file URLs or to a local zip (--zip) on machines without internet access. It writes SHA-256 checksums (verified in CI) and builds data/processed/heart_clean.csv. Unlike the common Cleveland-only approach (303 rows), **all four hospitals are combined**, which triples the sample size. This also brings in the realistic data-quality problems described below.

3.2 Cleaning

- '?' is read as missing. The 0-4 severity score num becomes a binary target (num > 0).
- Physiologically impossible zeros are converted to missing: chol = 0 (all 123 Swiss rows) and trestbps = 0.
- Out-of-range numerics and undocumented category codes are set to missing. Two exact duplicate rows are dropped.
- No rows are dropped for missing features. Imputation happens inside the model pipeline, so serving uses identical logic.

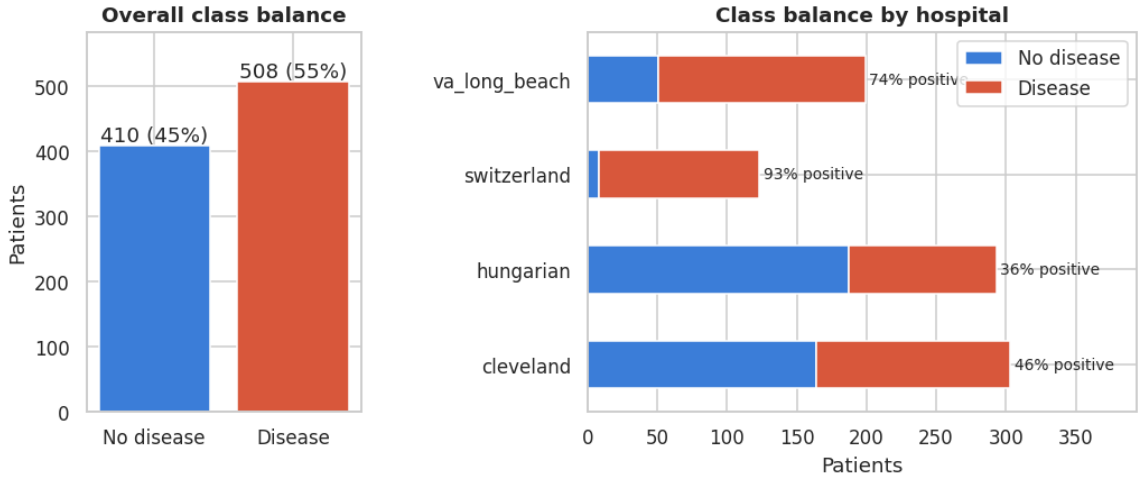


Figure 2 - Class balance overall and by hospital

3.3 Findings

The classes are balanced overall (55% positive), but the positive rate varies from 36% (Hungary) to 93% (Switzerland), so the train/test split is **stratified on hospital × target**. Missingness is strongly site-dependent: ca is 99% missing outside Cleveland, and thal and slope are mostly missing in Hungary and VA (Figure 3). Any 'was missing' signal therefore partly encodes the hospital rather than the patient's physiology.

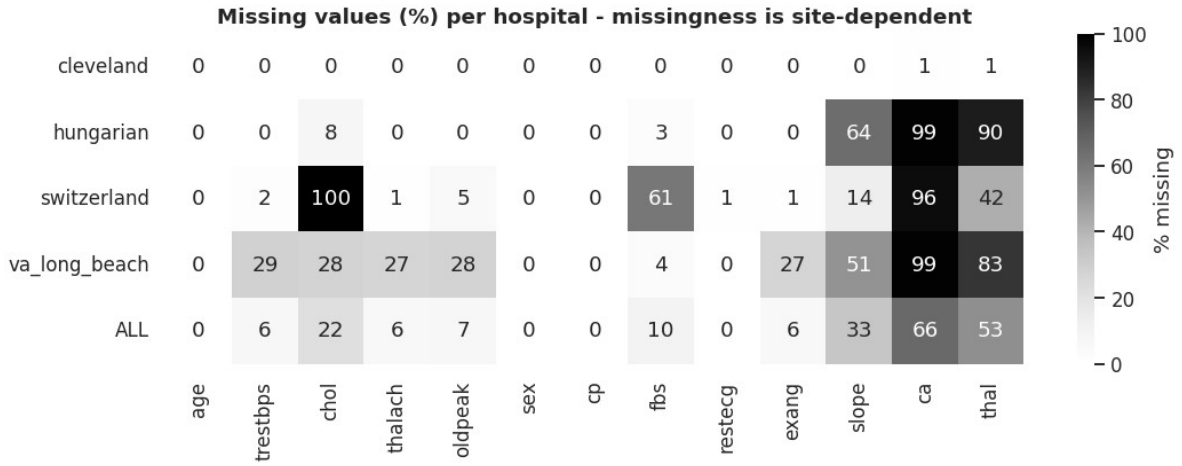


Figure 3 - Missing values (%) per hospital

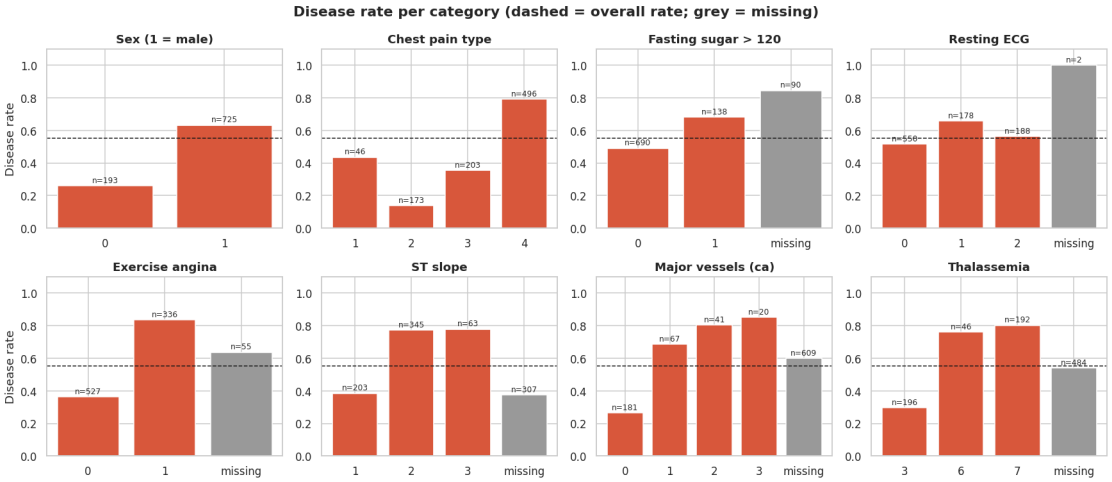


Figure 4 - Disease rate per category (grey = missing level)

The strongest univariate signals are asymptomatic chest pain (cp = 4, 79% positive), exercise-induced angina, ST depression (oldpeak), male sex, a lower maximum heart rate and the number of blocked vessels (ca). Maximum heart rate falls with age (Figure 6), which motivated the engineered feature **hr_reserve_pct = thalach / (220 – age)**: the share of age-predicted maximum heart rate the patient reached. Cholesterol and resting blood pressure have long right tails (outliers up to 603 mg/dl), which motivated a robust scaler.

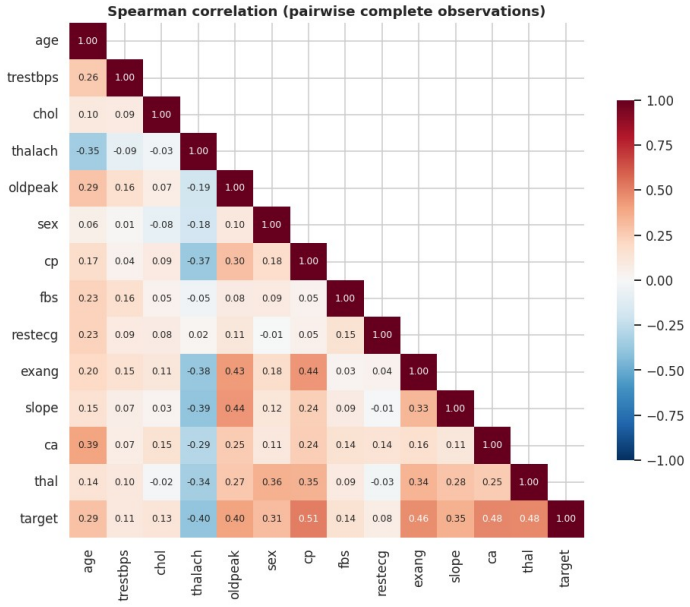


Figure 5 - Spearman correlation heatmap

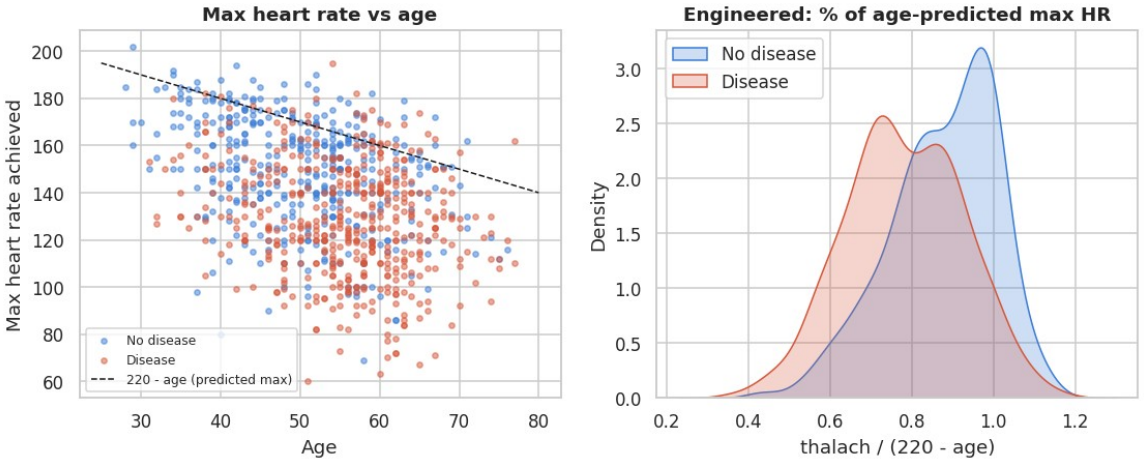


Figure 6 - Max heart rate vs age, and the engineered feature

4. Feature engineering and model development

4.1 Feature pipeline

Step	Columns	Choice and reason
ClinicalFeatures	age, thalach, oldpeak	Adds hr_reserve_pct and age_x_oldpeak (domain features)
Numeric	age, trestbps, chol, thalach, oldpeak + 2 engineered	Median imputation, then RobustScaler (outlier-tolerant)
Categorical	sex, cp, fbs, restecg, exang, slope, ca, thal	Missing becomes its own level (-1), then one-hot encoding; unknown levels are ignored at serving time
Missing indicators	numeric	Off by default (site proxy); tested as an ablation

4.2 Models, tuning and selection

Three model families were tuned with GridSearchCV (stratified 5-fold, shuffle, seed 42) on the 80% training split, 72 configurations in total (params.yaml). **Selection uses mean cross-validated ROC-AUC only**. The 20% hold-out set is used once, to report final performance.

Model	Best params	CV ROC-AUC	CV acc.	CV prec.	CV recall	Train-CV gap	Test ROC-AUC
Logistic Regression (L1)	C=0.3, class_weight=None, penalty=l1	0.905 ± 0.014	0.835	0.853	0.850	0.011	0.899
Random Forest	max_depth=6, max_features=sqrt, min_samples_leaf=3	0.895 ± 0.021	0.823	0.836	0.847	0.066	0.888
HistGradientBoosting	l2_regularization=1, learning_rate=0.03, max_depth=2, min_samples_leaf=10	0.891 ± 0.020	0.820	0.830	0.852	0.056	0.894

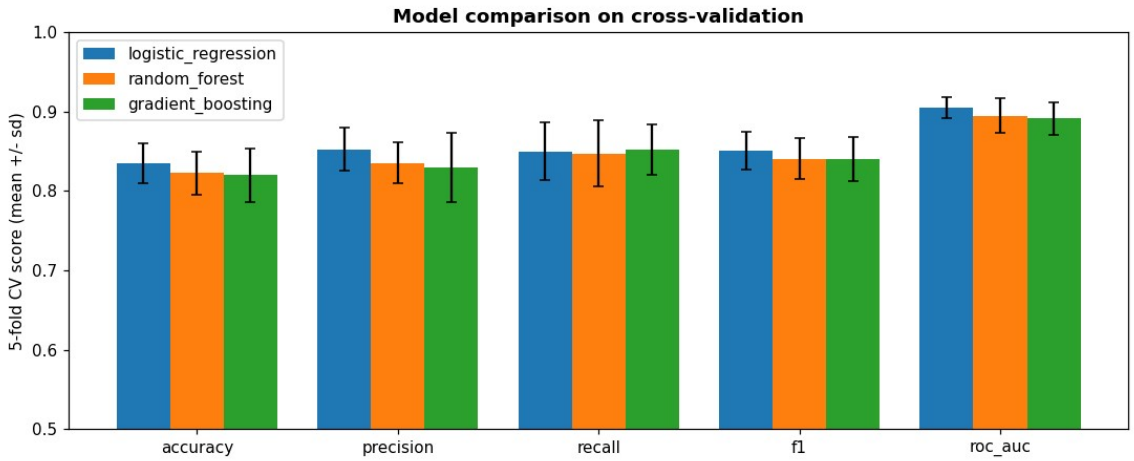


Figure 7 - Cross-validated comparison (mean ± sd over 5 folds)

Why logistic regression won. It has the highest CV ROC-AUC, the lowest fold-to-fold variance and a train/validation gap about 6x smaller than the tree ensembles, which overfit the 734-row training set. It also produces well-calibrated probabilities (Brier 0.123) and coefficients a clinician can read. Its L1 penalty selected C = 0.3, which shrinks weak one-hot levels to zero.

4.3 Decision threshold

For a screening tool, a missed patient costs more than a false alarm. The threshold therefore maximises **F2** (recall weighted 2x) on out-of-fold predictions, subject to a false-alarm budget of specificity ≥ 0.70. The result is

0.39. On the test set this gives recall 0.912 (9 of 102 sick patients missed) at specificity 0.732. The threshold is stored in the model metadata and can be overridden through a ConfigMap without rebuilding the image.

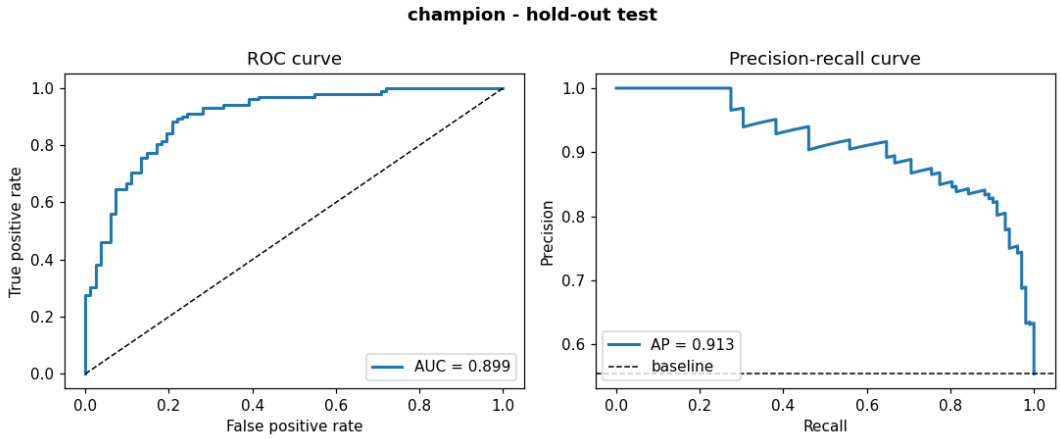


Figure 8 - Champion ROC and precision-recall curves on the hold-out test set

4.4 Robustness checks

Hospital (test)	n	Accuracy	Recall	ROC-AUC
cleveland	61	0.885	0.964	0.960
hungarian	58	0.776	0.762	0.860
switzerland	25	0.920	0.957	0.761
va_long_beach	40	0.775	0.933	0.697

Performance is strongest where all features are recorded (Cleveland, ROC-AUC 0.96) and weakest where ca, thal and chol are mostly absent (VA and Switzerland, both small and imbalanced). This is a real limitation of the data and is documented rather than hidden. The **missing-indicator ablation** gave out-of-fold ROC-AUC 0.9045 with indicators vs 0.9044 without. The difference is negligible, so the simpler model, which depends less on site artefacts, was kept. Permutation importance ranks cp, ca, fbs, slope, oldpeak and sex highest.

5. Experiment tracking (MLflow)

Every training run creates a nested hierarchy in the heart-disease-risk experiment (SQLite backend, local artifact store):

- **training-pipeline-<timestamp>** (parent): seed, CV setup, data SHA-256, row counts, git commit, library versions; artifacts: cleaned dataset, params.yaml, all EDA figures.
  - **model:<family>** (3 runs): best hyper-parameters, CV mean and sd for 5 metrics, train-CV gap, test metrics, ROC/PR, confusion and calibration plots, full cv_results CSV, and the logged sklearn model with signature and input example.
  - **grid:<family>:NNN** (72 runs): one run per hyper-parameter candidate with its CV metrics, which gives the parallel-coordinates view in Figure 10.
  - **ablation:missing_indicators** and **champion**: tuned threshold, per-hospital metrics, permutation importance. The champion is registered as heart-disease-classifier v1 with alias champion and a threshold tag.

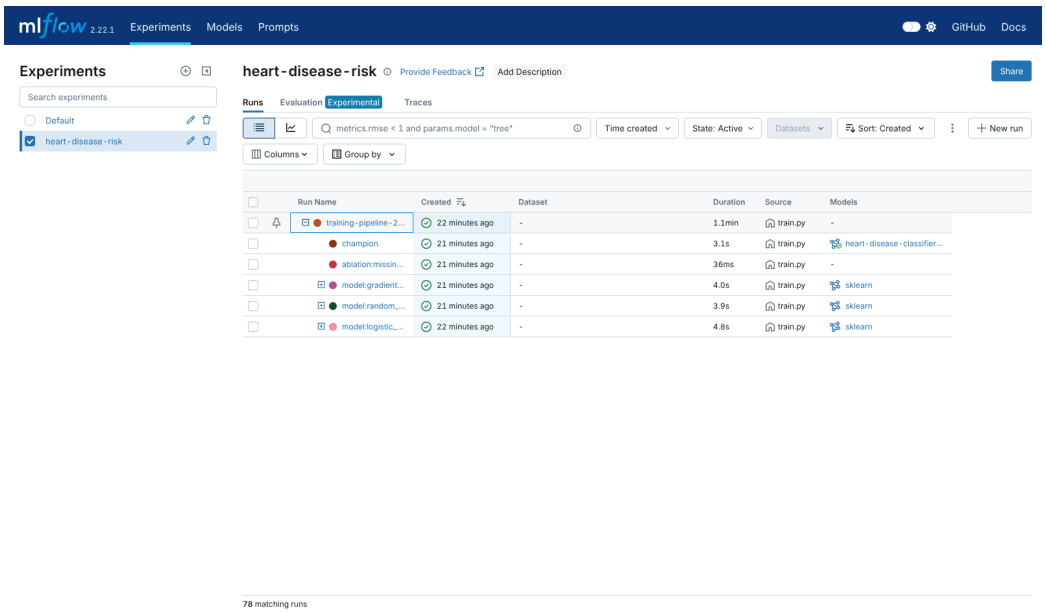


Figure 9 - MLflow run hierarchy (78 runs) and registered model

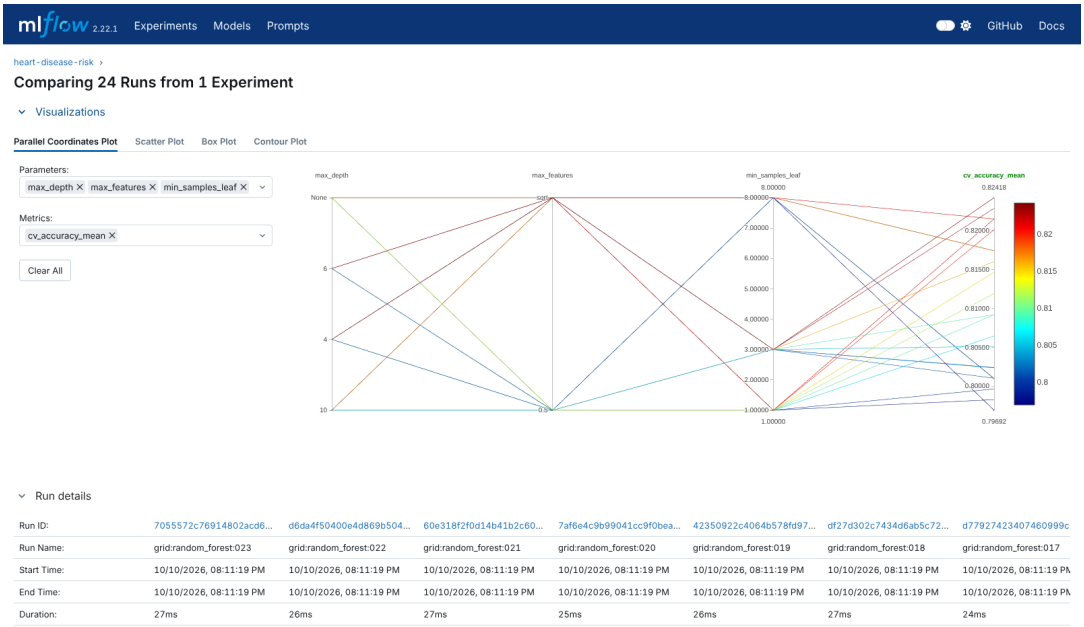


Figure 10 - Random forest grid: hyper-parameters vs CV score (24 child runs)

6. Model packaging and reproducibility

- **Formats:** models/model.joblib (the complete preprocessing and model Pipeline) plus model_metadata.json (version = package version + data hash, threshold, best params, CV and test metrics, git commit, training time). The model is also stored in MLflow format with code_paths, so mlflow.pyfunc.load_model('models:/heart-disease-classifier@champion') works in a new environment. Notebook 03 asserts that both artefacts give identical probabilities.
- **Environment:** requirements.txt is fully pinned (training, tracking, tests). requirements-serve.txt is the minimal runtime used by Docker. A pyproject.toml makes the code an installable package.
- **Determinism:** fixed seeds, checksummed raw data, the data hash logged and embedded in the model version, and every setting in params.yaml. Re-running the full pipeline in a fresh virtual environment (Prayogshala notebook dry run) reproduced identical test metrics and the same model version 1.0.0+327a95bf.

7. CI/CD pipeline and automated testing

7.1 Tests

Test module	What it guards
test_data.py (12)	Loading all 4 sites, target mapping, sentinel zeros, invalid codes, duplicates, schema and value ranges, input not mutated
test_model.py (13)	Engineered features, NaN-free preprocessing, unseen categories, model learns signal (AUC > 0.8), probabilities valid, reproducibility, threshold policy, params.yaml
test_api.py (17)	/predict contract, confidence, high vs low risk ordering, optional fields, 422 validation, batch, /metrics, dashboard, 503 when the model is not loaded, threshold override
test_configs.py (6)	Kubernetes YAML, probe/port contract with the Dockerfile, monitoring copies in sync, dashboard queries, alert rules, CI job order
test_pipeline_smoke.py (2)	EDA figures and a miniature end-to-end training run with an isolated MLflow store (all run types present)

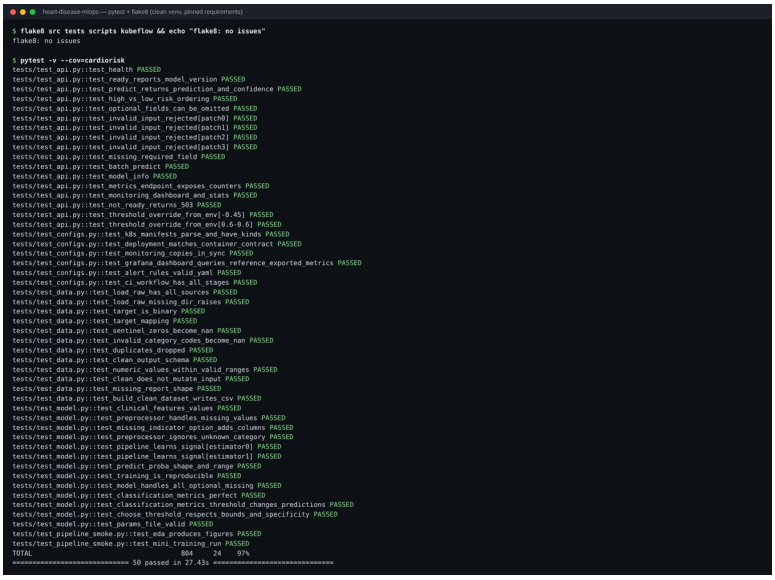


Figure 11 - flake8 and pytest in a clean virtual environment: 50 passed, 97% coverage

7.2 GitHub Actions workflow

.github/workflows/ci-cd.yml runs on every push and pull request. Each job depends on the previous one, so any failure stops the pipeline. Shell steps use pipefail so that errors are not hidden by tee.

Job	Steps	Artifacts / logs
lint	flake8 on src, tests, scripts, kubeflow	flake8-report.txt
test	clean pip install from requirements.txt, dataset checksum verification, pytest with JUnit and coverage	junit.xml, coverage.xml, HTML coverage, pytest.log, summary table
train	data script, EDA, MLflow training, quality gate (test ROC-AUC ≥ 0.85 or the job fails)	model, reports and figures, mlflow.db + mlartifacts, metrics table
docker	build with git sha label, run, wait for /ready, smoke-test /predict (asserts classes), batch, 422 check, /metrics, health status, push to GHCR on main	build log, container log, prediction JSON, image tar
deploy-kind	kind cluster, load image, kubectl apply -k, rollout status, port-forward, /ready and /predict, 100-request load test	kubectl resources, describe, pod logs

> CI evidence: after pushing the repository, add screenshots of the Actions run graph and job summaries here (screenshots/02_ci_pipeline.png, 02b_ci_summary.png). The workflow was validated locally (YAML structure test, every command run by hand), but it has not yet run on GitHub.

8. Containerisation and deployment

8.1 Docker image

A multi-stage Dockerfile builds a virtual environment that contains only the serving dependencies, then copies it into a slim runtime stage. The container runs as non-root uid 10001, defines a HEALTHCHECK on /health, carries OCI labels (git sha, build date), and starts uvicorn. The API accepts JSON and returns the class, probability, confidence (the probability of the predicted class), a risk band, and the optional inputs that were imputed. Pydantic rejects out-of-range or unknown fields with HTTP 422.

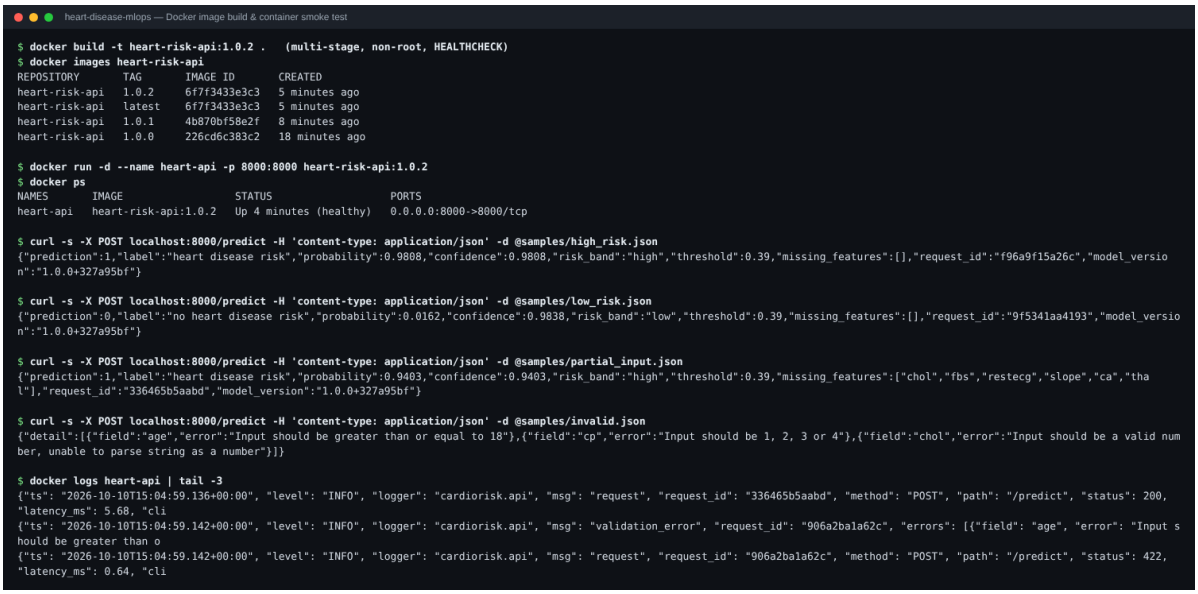


Figure 12 - Image build, healthy container and sample predictions (high, low, partial, invalid)

8.2 Kubernetes

Manifests are organised with kustomize (k8s/base, a k3s overlay that switches the ingress class to Traefik, and k8s/monitoring). The Deployment runs 2 replicas with a rolling update (maxUnavailable 0), startup, readiness and liveness probes, CPU and memory requests and limits, a read-only root filesystem with all capabilities dropped, Prometheus scrape annotations, and a preStop drain hook. The HPA scales between 2 and 6 pods at 70% CPU, and a PodDisruptionBudget keeps at least one pod available. The API is exposed through a LoadBalancer Service and an Ingress (heart-risk.local).

The deployment was verified on a local single-node Kubernetes cluster (k3s v1.31). The HPA scaled the API to 4 replicas under load. Verification found two real issues, both fixed and rolled out as new image versions: (1) an empty DECISION_THRESHOLD value in the ConfigMap crashed predictions, so it is now treated as unset and covered by a test; (2) about 1% of requests returned 502 during a rollout until the preStop drain hook was added. **After the fix, 1,111 of 1,111 requests sent through the Ingress during a rolling restart succeeded.**

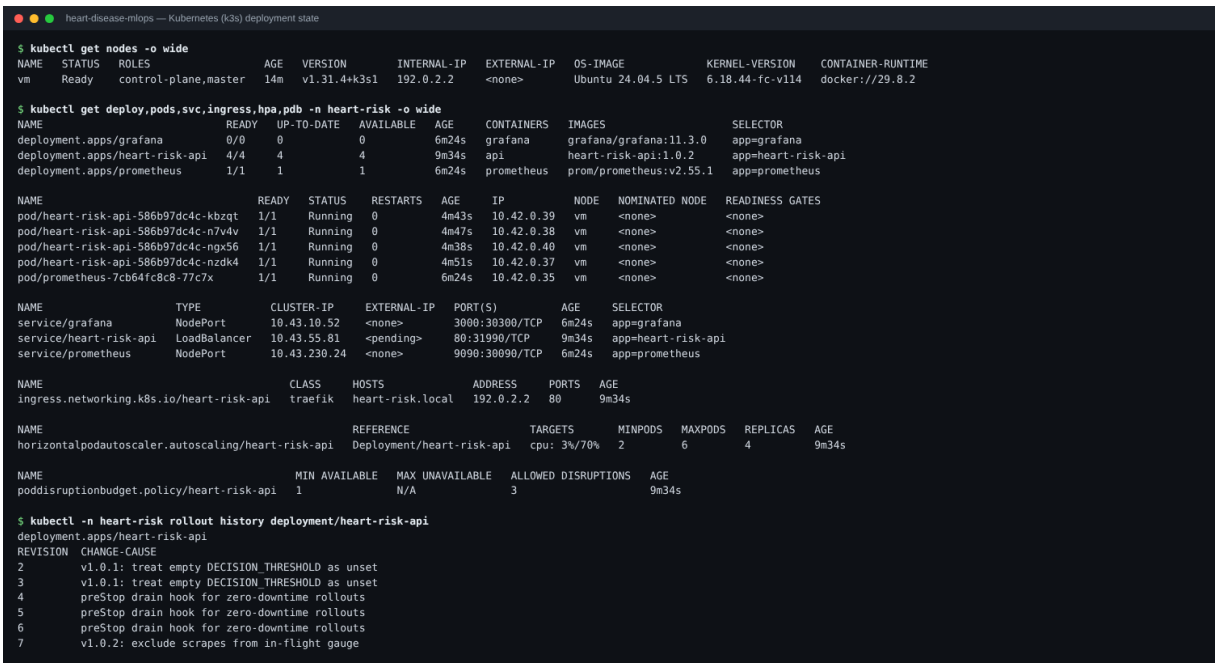


Figure 13 - Deployment, pods, Service, Ingress, HPA, PDB and rollout history

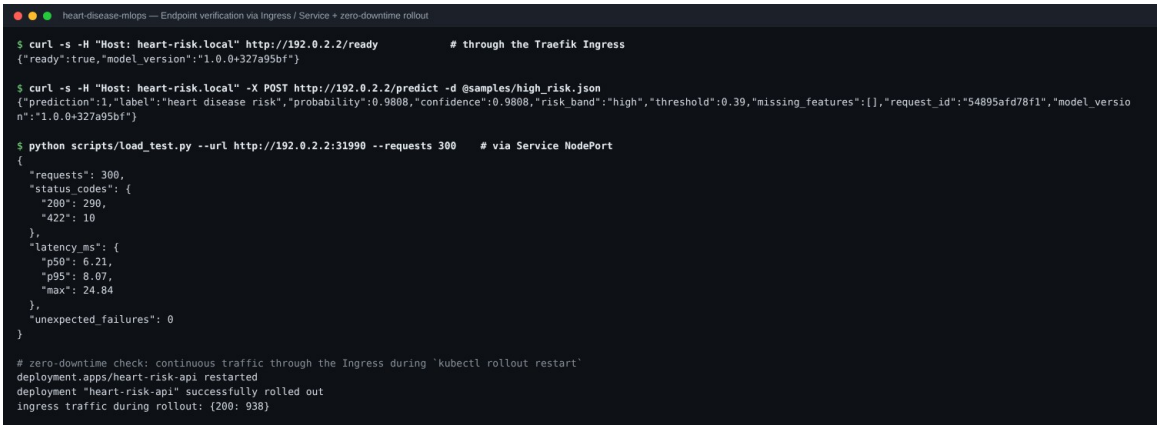


Figure 14 - Endpoint checks via Ingress and Service, load test and zero-downtime rollout

Access instructions: kubectl apply -k k8s/base (or k8s/overlays/k3s), then use the LoadBalancer IP (cloud, Docker Desktop, or minikube tunnel), the Ingress host heart-risk.local, or kubectl -n heart-risk port-forward svc/heart-risk-api 8080:80. For a public cloud (GKE, EKS or AKS) the same manifests apply with the GHCR image that CI pushes.

9. Monitoring and logging

- **Request logging:** every request produces one JSON log line with request_id (also returned in the x-request-id header), method, path, status, latency and client. Predictions and validation errors are logged as separate events.
- **Metrics (/metrics):** request counter by endpoint and status, latency histogram, in-flight gauge, predictions by outcome and risk band, predicted-probability histogram, imputed inputs per feature, validation errors, model info.
- **Prometheus** discovers every API pod through the Kubernetes API (RBAC-scoped Role) and scrapes it every 5 seconds. Five alert rules cover: target down, 5xx ratio > 5%, p95 latency > 250 ms, prediction drift (positive rate moves more than 25 points from the training base rate of 0.55), and imputation-heavy traffic. promtool validated the rules.
- **Grafana** dashboard (provisioned from JSON, 12 panels): targets up, traffic, p50/p95/p99 latency, risk-band mix, drift line against the training base rate, probability distribution and missing-input counts. All 15

dashboard queries were executed against the live Prometheus and returned data. The built-in /monitoring page gives a lightweight view without Grafana.

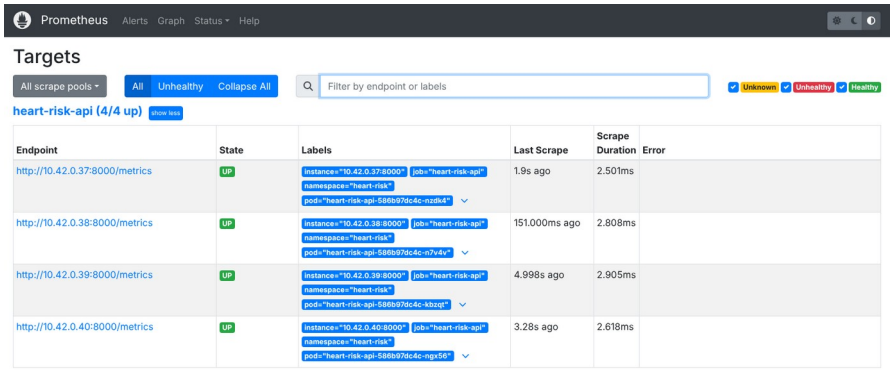


Figure 15 - Prometheus discovering and scraping all four API pods



Figure 16 - Per-pod request rate and p95 /predict latency under load

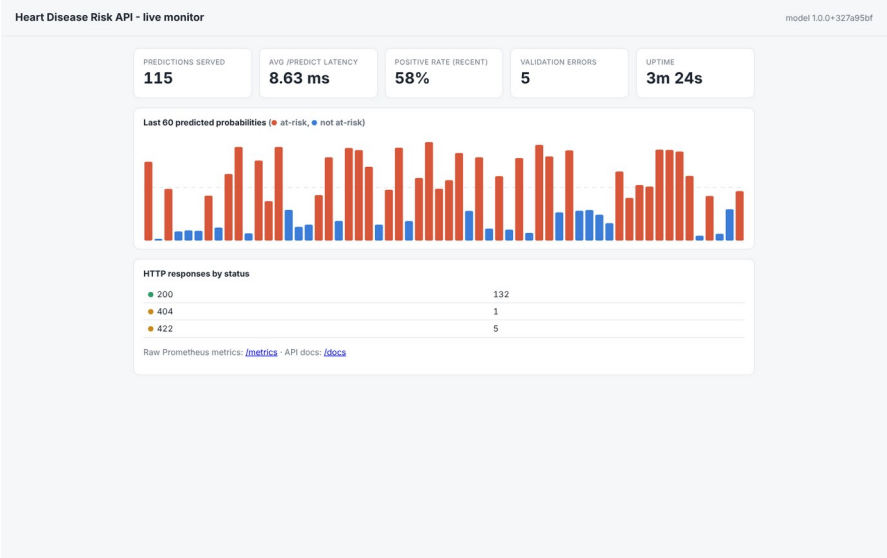


Figure 17 - Built-in live monitor (/monitoring)

10. Running in BITS Prayogshala

notebooks/00_prayogshala_end_to_end.ipynb reproduces the whole pipeline inside a Kubeflow notebook server: it locates the uploaded project (zip or git clone), installs the pinned requirements, verifies checksums, runs flake8 and pytest, runs EDA and MLflow training, queries the registry, starts the MLflow UI and the API as background processes, calls /predict, generates traffic and reads /metrics, checks Kubernetes permissions with kubectl auth can-i, and cleans up. A dry run in a brand-new virtual environment completed in about 2.5 minutes on 2 CPUs. kubeflow/pipeline.py (KFP v2, compiled and validated) runs the same steps as a pipeline: fetch-and-test, train-and-track (exports metrics and the model artefact), quality-gate, and a conditional publish-model step.

11. Limitations and next steps

- The data is small (918 patients) and decades old. Performance is weaker for hospitals with heavily missing ca/thal. This is a screening aid, not a diagnostic tool.
- Drift detection currently watches the output distribution. A next step is feature-level drift (e.g. PSI per input) computed from logged requests, plus scheduled re-training through the Kubeflow pipeline.
- MLflow uses a local SQLite store. A shared tracking server with object storage would suit a team setting.

12. Deliverables and links

Deliverable	Location
Code repository	https://github.com/<your-github-username>/heart-disease-mlops
Dataset + download script	data/raw, data/processed/heart_clean.csv, scripts/download_data.py
Notebooks	notebooks/00_prayogshala_end_to_end, 01_eda, 02_training, 03_inference
Tests / CI	tests/, .github/workflows/ci-cd.yml
Docker / Kubernetes	Dockerfile, docker-compose.yml, k8s/base, k8s/overlays/k3s, k8s/monitoring
Monitoring	monitoring/ (Prometheus config, alert rules, Grafana dashboard)
Screenshots	screenshots/
Video	report/VIDEO_SCRIPT.md (recording walkthrough)